

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет прикладной математики, информатики и механики

Кафедра программного обеспечения и
администрирования информационных систем

Операционные системы

Учебно-методическое пособие

Составители:

Г.Э. Вошинская, Е.М. Лещенко

Издательско-полиграфический центр
Воронежского государственного университета
2021

Утверждено научно-методическим советом факультета прикладной математики, информатики и механики от 15. 02. 2021

Рецензент доц. каф. МО ЭВМ ф-та ПММ к.ф.-м. н О.Д. Горбенко

Учебно-методическое пособие подготовлено на кафедре программного обеспечения и администрирования информационных систем факультета прикладной математики, информатики и механики Воронежского государственного университета.

Рекомендовано для студентов специальности 02.03.03 Математическое обеспечение и администрирование информационных систем для проведения лабораторных занятий по предмету «Операционные системы», со студентами 3 курса дневного отделения.

Содержание

Взаимодействие и синхронизация потоков	4
Задание 1 для самостоятельной работы.....	6
Перебор файлов с использованием FindFirstFile/FindNextFile и итераторов C# 2.0	8
Задание 2 для самостоятельного выполнения.....	14
Моментальные снимки	16
Задание 3 для самостоятельного выполнения.....	26
Список литературы	28

Взаимодействие и синхронизация потоков

Многопоточное приложение. Способы синхронизации

Приводимый ниже пример является многопоточным приложением, пара дополнительных потоков которого получают доступ к одному и тому же объекту (объекту синхронизации). Результаты воздействия образующих потоки операторов наглядно проявляются на экране консольного приложения.

```
using System;
using System.Threading;

namespace threads12
{
    class TextPresentation
    {
        public Mutex mutex;

        public TextPresentation()
        {
            mutex = new Mutex();
        }

        public void showText(string text)
        {
            int i;
            // Объект синхронизации в данном конкретном случае –
            // представитель класса TextPresentation. Для его обозначения
            // используется первичное выражение this.
            //1. Блокировка кода монитором (начало) // Monitor.Enter(this);
            //2. Критическая секция кода (начало) // lock(this) {
            //3. Блокировка кода мьютексом (начало)
            mutex.WaitOne();
            Console.WriteLine("\n" + (char)31 + (char)31 + (char)31 + (char)31);
            for (i = 0; i < 250; i++)
            {
                Console.Write(text);
            }
            Console.WriteLine("\n" + (char)30 + (char)30 + (char)30 + (char)30);
            mutex.ReleaseMutex();
            //3. Блокировка кода мьютексом (конец) //
            //2. Критическая секция кода (конец) // }
            //1. Блокировка кода монитором (конец) // Monitor.Exit(this);
        }
    }

    class threadsRunners
    {
        public static TextPresentation tp = new TextPresentation();
        public static void Runner1()
        {

```

```

        Console.WriteLine("thread_1 run!");
        Console.WriteLine("thread_1 - calling TextPresentation.showText");
        tp.showText("*");
        Console.WriteLine("thread_1 stop!");
    }

    public static void Runner2()
    {
        Console.WriteLine("thread_2 run!");
        Console.WriteLine("thread_2 - calling TextPresentation.showText");
        tp.showText("|");
        Console.WriteLine("thread_2 stop!");
    }

    static void Main(string[] args)
    {
        ThreadStart runner1 = new ThreadStart(Runner1);
        ThreadStart runner2 = new ThreadStart(Runner2);

        Thread th1 = new Thread(runner1);
        Thread th2 = new Thread(runner2);

        th1.Start();
        th2.Start();
    }
}

```

Рекомендации по недопущению блокировок потоков

- Соблюдать определенный порядок при выделении ресурсов.
- При освобождении выделенных ресурсов придерживаться обратного (reverse) порядка.
- Минимизировать время неопределенного ожидания выделяемого ресурса.
- Не захватывать ресурсы без необходимости и при первой возможности освобождать захваченные ресурсы.
- Захватывать ресурс только в случае крайней необходимости.
- В случае, если ресурс не удастся захватить, повторную попытку его захвата производить только после освобождения ранее захваченных ресурсов.
- Максимально упрощать структуру задачи, решение которой требует захвата ресурсов. Чем проще задача – тем на меньший период захватывается ресурс.

Задание 1 для самостоятельной работы

- I. Реализовать задачу «Поставщик-потребитель». Поставщик генерирует данные и отправляет их в общий буфер. Размер буфера ограничен. Потребитель забирает данные из буфера. Поставщик не может положить данные, если в буфере нет свободных мест. Потребитель не может взять данные, если буфер пуст. Поставщик и потребитель не могут одновременно работать с буфером.

Создать Windows-приложение. Каждый поставщик и каждый потребитель — поток. Буфер описан как собственный класс, который внутри может использовать соответствующий библиотечный контейнер. Средства синхронизации встроены в буфер в команды «положить» и «взять». На форме должны отображаться состояние буфера и состояние поставщика и потребителя, должна быть возможность приостановить и возобновить потоки.

Варианты заданий по представлению буфера(а), по средствам синхронизации(б), по задаче(с).

а. Представление буфера

- 1) Стек
- 2) Очередь

б. Средства синхронизации.

- 1) Критические секции (lock).
- 2) Семафоры и мьютексы.
- 3) Монитор(Enter-Exit).

с. Задача

1) Создать приложение с двумя дополнительными потоками писателей и двумя потоками читателей. Писатели в случайные моменты времени помещают записи, содержащие номер потока писателя и номер записи в буфер, читатели в случайные моменты времени удаляют записи, делая об этом сообщения. Все читатели и писатели используют один и тот же буфер.

2) Создать многопоточное приложение с одним потоком - читателем, удаляющим данные из буфера. Главный поток в случайные моменты времени порождает потоки - писатели, которые в случайные моменты времени помещают данные в буфер, если в структуре имеется свободное место, или самоуничтожаются с

соответствующим сообщением. Все читатели и писатели используют один и тот же буфер.

3) Создать многопоточное приложение с одним потоком - писателем, который в случайные моменты времени помещает данные в буфер и сообщает об этом. Главный поток в случайные моменты времени порождает потоки - читатели, которые в случайные моменты времени удаляют данные из буфера с соответствующим сообщением. Каждый поток – читатель завершается после удаления заданного числа данных. Все читатели и писатели используют один и тот же буфер.

4) Создать многопоточное приложение, в котором главный поток в случайные моменты времени порождает либо поток - читатель, который в случайные моменты времени удаляет данные из буфера с соответствующим сообщением, либо поток - писатель, который в случайные моменты времени помещает данные в буфер и сообщает об этом. Каждый поток – читатель завершается после удаления заданного числа данных. Каждый поток – писатель завершается после занесения заданного числа данных. Все читатели и писатели используют один и тот же буфер.

5) Создать многопоточное приложение, в котором главный поток создает три потока: первый поток создает сообщение и помещает его в первый буфер, второй поток забирает сообщение из первого буфера, добавляет к сообщению свою информацию и передает сообщение во второй буфер, третий поток забирает сообщение из второго буфера и добавляет к нему свою информацию и сообщает об этом.

6) Создать приложение с двумя дополнительными потоками писателей и двумя потоками читателей. Писатели в случайные моменты времени помещают записи, содержащие номер потока писателя и номер записи в буфер, читатели в случайные моменты времени удаляют записи, делая об этом сообщения. Каждая пара читатель — писатель использует свой буфер.

7) Создать многопоточное приложение с одним потоком - читателем, удаляющим данные из буфера. Главный поток в случайные моменты времени порождает потоки - писатели, которые в случайные моменты времени помещают данные в буфер, если в

структуре имеется свободное место, или самоуничтожаются с соответствующим сообщением. Каждая пара читатель — писатель использует свой буфер.

8) Создать многопоточное приложение с одним потоком - писателем, который в случайные моменты времени помещает данные в буфер и сообщает об этом. Главный поток в случайные моменты времени порождает потоки - читатели, которые в случайные моменты времени удаляют данные из буфера с соответствующим сообщением. Каждый поток – читатель завершается после удаления заданного числа данных. Каждая пара читатель — писатель использует свой буфер.

9) Создать многопоточное приложение, в котором главный поток в случайные моменты времени порождает либо поток - читатель, который в случайные моменты времени удаляет данные из буфера с соответствующим сообщением, либо поток - писатель, который в случайные моменты времени помещает данные в буфер и сообщает об этом. Каждый поток – читатель завершается после удаления заданного числа данных. Каждый поток – писатель завершается после занесения заданного числа данных. Каждая пара читатель — писатель использует свой буфер.

10) Создать приложение с двумя дополнительными потоками писателем и читателем. Писатели в случайные моменты времени помещают записи в буфер, читатели в случайные моменты времени считывают записи, добавляют свою информацию и ответ отправляют писателю. Писатель считывает ответ и сообщает об этом.

Перебор файлов с использованием FindFirstFile/FindNextFile и итераторов C# 2.0

Для поиска заданного каталога, системных каталогов, каталогов, заданных с помощью переменной окружения PATH, или списка каталогов, разделенных точкой с запятой, можно использовать функцию Win32 API SearchPath(). К сожалению, эта функция не выполняет поиск файла среди

подкаталогов данного каталога. Для поиска файлов в некотором каталоге и его подкаталогах можно использовать пару функций

```
IntPtr FindFirstFile(string lpFileName, out WIN32_FIND_DATA lpFindFileData);
```

и

```
bool FindNextFile(IntPtr hFindFile, out WIN32_FIND_DATA lpFindFileData);
```

Функция **FindFirstFile** создает дескриптор поиска и возвращает информацию **lpFindFileData** о первом файле, имя которого соответствует указанному образцу в **lpFileName** (указатель на символьную строку с нулем в конце, которая определяет правильный каталог или путь и имя файла, которое может содержать символы подстановки * и ?). Функция **FindNextFile** продолжает поиск в том же каталоге, получая от функции **FindFirstFile** дескриптор поиска **hFindFile**, и возвращая информацию об очередном файле в **lpFindFileData**.

Структура **WIN32_FIND_DATA** содержит информацию о найденном файле или подкаталоге и определяется следующим образом:

```
private struct WIN32_FIND_DATA
{
    public FileAttributes dwFileAttributes;
    public FILETIME ftCreationTime;
    public FILETIME ftLastAccessTime;
    public FILETIME ftLastWriteTime;
    public int nFileSizeHigh;
    public int nFileSizeLow;
    public int dwReserved0;
    public int dwReserved1;
}
```

Значения полей записи WIN32_FIND_DATA

Поле	Значение
dwFileAttributes	Атрибуты найденного файла
ftCreationTime	Время создания файла
ftLastAccessTime	Время последнего доступа к файлу
ftLastWriteTime	Время последней модификации файла
nFileSizeHigh	Старшие разряды (старшее двойное слово DWORD) размера файла в байтах. Если размер файла не превышает MAXWORD, то это значение равно 0

nFileSizeLow	Младшие разряды(младшее двойное слово DWORD) размера файла в байтах
dwReserved0	В данный момент не используется - зарезервировано
dwReserved1	В данный момент не используется - зарезервировано
cFileName	Имя файла в виде строки с ограничивающим нуль – символом
cAlternateFileName	Имя файла в формате 8.3, усечение длинного имени.

Параметр dwFileAttributes определяет атрибуты файла.

Атрибут	Значение	Описание
faReadOnly	\$01	Файл, предназначенный только для чтения
faHidden	\$02	Скрытый файл
faSysFile	\$04	Системный файл
faVolumeID	\$08	Метка тома
faDirectory	\$10	Каталог
faArchive	\$20	Архивный файл
faAnyFile	\$3F	Любой файл

Для установки атрибутов можно использовать функцию
BOOL SetFileAttributes (LPCTSTR, DWORD);

Пример программы, осуществляющей перебор файлов в некотором каталоге, включая все его подкаталоги.

Перебор файлов с использованием C#.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Runtime.InteropServices;
using System.IO;
using FILETIME = System.Runtime.InteropServices.ComTypes.FILETIME;
using System.ComponentModel;

namespace FindFiles
{
    public class RsdnDirectory
```

```

{
    /// <summary>
    /// Формирует путь требуемый функцией FindFirstFile.
    /// </summary>
    private static string MakePath(string path)
    {
        return Path.Combine(path, "*");
    }

    /// <summary>
    /// Возвращает список файлов или каталогов находящихся по заданному
    ///пути path.
    /// </summary>
    /// <param name="path">Путь для которого нужно вернуть
    ///список.</param>
    /// <param name="isGetDirs">
    /// Если true - функция возвращает список каталогов, иначе файлов.
    /// </param>
    /// <returns>Список файлов или каталогов.</returns>
    private static IEnumerable<string> GetInternal(string path, bool
        isGetDirs)
    {
        // Структура в которую функции FindFirstFile и FindNextFile
        // возвращают информацию о текущем файле.
        WIN32_FIND_DATA findData;
        // Получаем информацию о текущем файле и дескриптор
        // перечислителя.
        // Этот дескриптор требуется передавать функции FindNextFile
        // для получения следующих файлов.
        IntPtr findHandle = FindFirstFile(MakePath(path),
            out findData);

        // Если произошла ошибка, то
        // нужно вынуть информацию об ошибке и перепаковать ее в
        // исключение.
        if (findHandle == INVALID_HANDLE_VALUE)
            throw new Win32Exception(Marshal.GetLastWin32Error());

        try
        {
            do
            {
                if (isGetDirs ? (findData.dwFileAttributes &
                    FileAttributes.Directory) != 0
                    : (findData.dwFileAttributes &
                    FileAttributes.Directory) == 0)
                    yield return findData.cFileName;
                while (FindNextFile(findHandle, out findData));
            }
            finally
            {
                FindClose(findHandle);
            }
        }
    }
}

```

```

/// <summary>
/// Возвращает список файлов для некоторого пути.
/// </summary>
/// <param name="path">
/// Каталог для которого нужно получить список файлов.
/// </param>
/// <returns>Список файлов каталога.</returns>
public static IEnumerable<string> GetFilse(string path)
{
    return GetInternal(path, false);
}

/// <summary>
/// Возвращает список каталогов для некоторого пути. Функция не
/// перебирает вложенные подкаталоги!
/// </summary>
/// <param name="path">
/// Каталог для которого нужно получить список подкаталогов.
/// </param>
/// <returns>Список файлов каталога.</returns>
public static IEnumerable<string> GetDirectories(string path)
{
    return GetInternal(path, true);
}

/// <summary>
/// Функция возвращает список относительных путей ко всем
/// подкаталогам
/// (в том числе и вложенным) заданного пути.
/// </summary>
/// <param name="path">Путь для которого унжно получить
/// подкаталоги.</param>
/// <returns>Список подкатлогов.</returns>
public static IEnumerable<string> GetAllDirectories(string path)
{
    // Сначала перебираем подкаталоги первого уровня вложенности...
    foreach (string subDir in GetDirectories(path))
    {
        // игнорируем имя текущего каталога и родительского.
        if (subDir == ".." || subDir == ".")
            continue;

        // Комбинируем базовый путь и имя подкаталога.
        string relativePath = Path.Combine(path, subDir);

        // возвращаем пользователю относительный путь.
        yield return relativePath;

        // Создаем рекурсивно итератор для каждого подкаталога и...
        // возвращаем каждый его элемент в качестве элементов
        // текущего итератора.
        // Этот прием позволяет обойти ограничение

```

```

        // итераторов C# 2.0, связанное с невозможностью вызовов
        // "yield return" из функций вызываемых из
        // функции итератора. К сожалению это приводит к созданию
        // временного вложенного итератора на каждом шаге рекурсии,
        // но затраты на создание такого объекта относительно
        // невелики, а удобство очень даже ощутимо.
        foreach (string subDir2 in GetAllDirectories(relativePath))
            yield return subDir2;
    }
}

#region Импорт из kernel32

private const int MAX_PATH = 260;

[Serializable]
[StructLayout(LayoutKind.Sequential, CharSet = CharSet.Auto)]
[BestFitMapping(false)]
private struct WIN32_FIND_DATA
{
    public FileAttributes dwFileAttributes;
    public FILETIME ftCreationTime;
    public FILETIME ftLastAccessTime;
    public FILETIME ftLastWriteTime;
    public int nFileSizeHigh;
    public int nFileSizeLow;
    public int dwReserved0;
    public int dwReserved1;
    [MarshalAs(UnmanagedType.ByValTStr, SizeConst = MAX_PATH)]
    public string cFileName;
    [MarshalAs(UnmanagedType.ByValTStr, SizeConst = 14)]
    public string cAlternate;
}

[DllImport("kernel32", CharSet = CharSet.Auto, SetLastError = true)]
private static extern IntPtr FindFirstFile(string lpFileName,
    out WIN32_FIND_DATA lpFindFileData);

[DllImport("kernel32", CharSet = CharSet.Auto, SetLastError = true)]
private static extern bool FindNextFile(IntPtr hFindFile,
    out WIN32_FIND_DATA lpFindFileData);

[DllImport("kernel32.dll", SetLastError = true)]
private static extern bool FindClose(IntPtr hFindFile);

private static readonly IntPtr INVALID_HANDLE_VALUE = new IntPtr(-1);

#endregion
}
}

```

Задание 2 для самостоятельного выполнения

Указание. Создать класс выполняющий задание для одного каталога. Поиск файлов реализовать с помощью функций `FindFirstFile()` и `FindNextFile()`. Главный поток обеспечивает реакцию формы. Вспомогательный поток запускает потоки для поиска файлов и ожидает от них результатов (использует для этого средства синхронизации).

1. Заданы два каталога. Сравнить, в каком из них больше вложенных каталогов. Вывести все.
2. Заданы два каталога. Определить, в каком из них больше подкаталогов, размер которых превышает заданное значение. В размер подкаталога включаются размеры его файлов-некаталогов.
3. Заданы два каталога. Проверить, есть ли в них каталоги с именем, совпадающим с именем какого-то из вложенных подкаталогов.
4. Заданы два каталога. Проверить, есть ли в них файлы, совпадающие (по названию). Вывести уникальные.
5. Заданы два каталога. Проверить, есть ли в них файлы, совпадающие (по названию). Вывести названия тех, для которых есть совпадения.
6. Заданы два каталога. Проверить, есть ли во втором файлы, созданные раньше, чем любой из первого каталога.
7. Заданы два каталога. Проверить, есть ли во втором файлы, созданные позже, чем любой из первого каталога.
8. Заданы два каталога. Сравнить их по количеству скрытых.
9. Заданы два каталога. Сравнить их по количеству Readonly.
10. Заданы два каталога. Сравнить их по количеству архивных.
11. Заданы два каталога. Сравнить общий объем содержащихся в каждом из них файлов.
12. Заданы два каталога. Для каждого из них для каждого каталога вывести имя самого большого файла.
13. Заданы два каталога. Для каждого из них найти, вывести и сравнить общее количество файлов, диапазон дат создания.
14. Заданы два каталога. В каждом из них вывести имена каталогов, в которых нет вложенных подкаталогов.
15. Заданы два каталога. В каждом из них найти файлы, у которых время последнего доступа и время создания совпадают. Сравнить их количество.
16. Заданы два каталога. Для каждого из них найти файлы, у которых дата модификации позже заданной даты. Сравнить их количество.
17. Заданы два каталога. Для каждого подсчитать количество файлов, размер которых превышает заданное значение. Сравнить их количество.

18. Заданы два каталога. Для каждого из них подсчитать количество каталогов, содержащих более чем m файлов. Сравнить их количество.
19. Заданы два каталога. Для каждого из них вывести те из них, в которых все файлы созданы позже указанной даты.
20. Заданы два каталога. Для каждого из них подсчитать количество каталогов, не содержащих скрытых файлов. Сравнить их количество.

Моментальные снимки

ToolHelp32 — это семейство функций и процедур, составляющих подмножество Win32 API, которые позволяют получить сведения о некоторых низкоуровневых аспектах работы ОС. В частности, сюда входят функции, с помощью которых можно получить информацию обо всех процессах, выполняющихся в системе в данный момент, а также потоках, модулях и кучах, принадлежащих каждому процессу.

Типы и определения функций ToolHelp32 размещаются в классе ToolHelp32, поэтому при работе с этими функциями нужно включить его имя в список инструкции

```
using System.Runtime.InteropServices.
```

Благодаря многозадачной природе среды Win32 такие объекты, как процессы, потоки, модули и т.п., постоянно создаются, разрушаются и модифицируются. И поскольку состояние компьютера непрерывно изменяется, системная информация, которая, возможно, будет иметь значение в данный момент, через секунду не будет актуальной. Например, предположим, что вы хотите написать программу для регистрации всех модулей, загруженных в систему. Поскольку операционная система в любое время может прервать выполнение потока, отработывающего вашу программу, чтобы предоставить какие-то кванты времени другому потоку в системе, модули теоретически могут создаваться и разрушаться даже в момент выборки информации о них.

В этой динамической среде имело бы смысл на мгновение заморозить систему, чтобы получить такую системную информацию. В ToolHelp32 не предусмотрено средств замораживания системы, но есть функция, с помощью которой можно сделать "снимок" системы в заданный момент времени. Эта функция — CreateToolhelp32Snapshot(), и ее объявление выглядит следующим образом:

```
static extern IntPtr CreateToolhelp32Snapshot([In]UInt32 dwFlags, [In]UInt32  
th32ProcessID);
```

Параметр dwFlags означает тип информации, подлежащий включению в моментальный снимок. Этот параметр может иметь одно из перечисленных в табл. значений.

```
internal enum SnapshotFlags : uint  
{  
    HeapList = 0x00000001,  
    Process = 0x00000002,  
    Thread = 0x00000004,  
    Module = 0x00000008,  
    Module32 = 0x00000010,  
    Inherit = 0x80000000,  
}
```



```

    All = 0x0000001F,
    NoHeaps = 0x40000000
}

```

Значение	Описание
Inherit	Означает, что дескриптор снимка будет наследуемым
All	Эквивалентно заданию значений HeapList, Process, Thread, Module, Module32, Inherit.
HeapList	Включает в снимок список куч заданного процесса Win32
Module	Включает в снимок список модулей заданного процесса Win32
Process	Включает в снимок список процессов Win32
Thread	Включает в снимок список потоков Win32

Функция `CreateToolhelp32Snapshot` возвращает дескриптор созданного снимка или `-1` в случае ошибки. Возвращаемый дескриптор работает подобно другим дескрипторам Win32 относительно процессов и потоков, для которых он действителен.

Следующий код создает дескриптор снимка, который содержит информацию обо всех процессах, потоках, модулях и кучах загруженных в настоящий момент:

```

IntPtr handleToSnapshot = IntPtr.Zero;
try
{
    handleToSnapshot =
        CreateToolhelp32Snapshot((uint)SnapshotFlags.All, 0);

    catch
    {
        throw new ApplicationException(string.Format("Failed with
win32 error code {0}", Marshal.GetLastWin32Error()));
    }
}

```

Для освобождения связанных с ним ресурсов используйте функцию `CloseHandle()`

Обработка информации о процессах

Имея дескриптор снимка, содержащий информацию о процессах, можно воспользоваться двумя функциями ToolHelp32, которые позволяют последовательно просмотреть сведения обо всех процессах в системе. Функции Process32First() и Process32Next() определены следующим образом:

```
static extern bool Process32First([In]IntPtr hSnapshot,  
                                ref ProcessEntry32 lppe);  
  
static extern bool Process32Next([In]IntPtr hSnapshot,  
                                ref ProcessEntry32 lppe);
```

Первый параметр у обеих функций является дескриптором снимка, возвращаемым функцией CreateToolhelp32Snapshot(). Второй параметр lppe, представляет собой структуру PROCESSENTRY32, которая передается по ссылке. По мере прохождения по элементам перечисления функции будут заполнять эту запись информацией о следующем процессе.

Структура PROCESSENTRY32 определяется так:

```
public struct PROCESSENTRY32  
{  
    public uint dwSize; // размер записи  
    public uint cntUsage; // счетчик ссылок процесса  
    public uint th32ProcessID; // идентификационный номер процесса  
    public IntPtr th32DefaultHeapID; //  
    public uint th32ModuleID; // идентифицирует модуль связанный  
                            // с процессом  
    public uint cntThreads; // кол-во потоков в данном процессе  
    public uint th32ParentProcessID; // идентификатор родительского  
                            // процесса  
    public int pcPriClassBase; // базовый приоритет процесса  
    public uint dwFlags; // зарезервировано  
    public string szExeFile; // путь к exe файлу или драйверу  
                            // связанному с этим процессом  
}
```

- В поле dwSize содержится размер записи ProcessEntry32. До использования этой записи поле dwSize должно быть инициализировано значением `SizeOf(typeof(PROCESSENTRY32))`.
- В поле cntUsage хранится значение счетчика ссылок процесса. Когда это значение станет равным нулю, операционная система выгрузит процесс.
- В поле th32ProcessID содержится идентификационный номер процесса.

- Поле `th32DefaultHeapID` предназначено для хранения идентификатора (ID) для кучи процесса, действующей по умолчанию. Этот ID имеет значение только для функций `ToolHelp32`, и его нельзя использовать с другими функциями `Win32`.
- Поле `thModuleID` идентифицирует модуль, связанный с процессом. Это поле имеет значение только для функций `ToolHelp32`.
- По значению поля `cntThreads` можно судить о том, сколько потоков начало выполняться в данном процессе.
- Поле `th32ParentProcessID` идентифицирует родительский процесс для данного процесса.
- В поле `pcPriClassBase` хранится базовый приоритет процесса. Операционная система использует это значение для управления работой потоков.
- Поле `dwFlags` зарезервировано.
- В поле `szExeFile` содержится строка с ограничивающим нуль-символом, которая представляет собой путь и имя файла EXE-программы или драйвера, связанного с данным процессом.

После создания снимка, содержащего информацию о процессах, для опроса данных по каждому процессу следует вызвать сначала функцию `Process32First()`, а затем вызывать функцию `Process32Next()` до тех пор, пока она не вернет значение `False`.

```
IntPtr handleToSnapshot = IntPtr.Zero;
try
{
    PROCESSENTRY32 procEntry = new PROCESSENTRY32 ();
    procEntry.dwSize = (UInt32)Marshal.SizeOf(typeof(ProcessEntry32));
    handleToSnapshot = CreateToolhelp32Snapshot((uint)SnapshotFlags.All,
0);
    if (Process32First(handleToSnapshot, ref procEntry))
    {
        do
        {
            yield return procEntry;
        } while (Process32Next(handleToSnapshot, ref procEntry));
    }
    else
    {
        throw new ApplicationException(string.Format
("Failed with win32 error code {0}",
Marshal.GetLastWin32Error()));
    }
}
```

```
finally
{
    CloseHandle(handleToSnapshot);
}
```

Обработка информации о потоках

Для составления списка потоков некоторого процесса в ToolHelp32 предусмотрены две функции, которые аналогичны функциям, предназначенным для регистрации процессов: Thread32First() и Thread32Next(). Эти функции объявляются следующим образом:

```
public static extern bool Thread32First(IntPtr hSnapshot, ref THREADENTRY32
    lpte);

public static extern bool Thread32Next(IntPtr hSnapshot, ref THREADENTRY32 lpte);
```

Помимо обычного параметра hSnapshot, этим функциям также передается по ссылке параметр типа THREADENTRY32. Как и в случае функций, работающих с процессами, каждая из них заполняет структуру THREADENTRY32, объявление которой имеет вид

```
public struct THREADENTRY32
{
    public uint dwSize;
    public uint cntUsage;
    public uint th32ThreadID;
    public uint th32OwnerProcessID;
    public int tpBasePri;
    public int tpDeltaPri;
    public uint dwFlags;
}
```

- Поле *dwSize* определяет размер структуры, и поэтому оно должно быть инициализировано значением SizeOf(typeof(HEAPLIST32)) до её использования.
- В поле *cntUsage* содержится счетчик ссылок данного потока. При обнулении этого счетчика поток выгружается операционной системой.
- Поле *th32ThreadID* представляет собой идентификационный номер потока, который имеет значение только для функций ToolHelp32.
- В поле *th32OwnerProcessID* содержится идентификатор (ID) процесса, которому принадлежит данный поток. Этот ID можно использовать с другими функциями Win32.
- Поле *tpBasePri* представляет собой базовый класс приоритета потока. Это значение одинаково для всех потоков данного

процесса. Возможные значения этого поля обычно лежат в диапазоне от 4 до 24. Описания этих значений приведены ниже.

Допустимые значения класса приоритета потоков

Значения	Описание
4	Ожидающий
8	Нормальный
13	Высокий
24	Реальное время

- Поле *tpDeltaPri* представляет собой дельта-приоритет (разницу), определяющий величину отличия реального приоритета от значения *tpBasePri*. Это число со знаком, которое в сочетании с базовым классом приоритета отображает общий приоритет потока.
- Поле *dwFlags* в данный момент зарезервировано и не должно использоваться.

Обработка информации о модулях

Опрос модулей выполняется практически так же, как опрос процессов или потоков. Для этого в ToolHelp32 предусмотрены две функции: *Module32First()* и *Module32Next()*, которые определяются следующим образом:

```
public static extern bool Module32First(IntPtr hSnapshot,
                                       ref MODULEENTRY32 lpme);
public static extern bool Module32Next(IntPtr hSnapshot,
                                       ref MODULEENTRY32 lpme);
```

Первым параметром в обеих функциях является дескриптор снимка, а вторым параметром — структура *MODULEENTRY32*. Ее определение имеет следующий вид:

```
public struct MODULEENTRY32
{
    public uint dwSize;
    public uint th32ModuleID;
    public uint th32ProcessID;
    public uint GblcntUsage;
    public uint ProccntUsage;
    public IntPtr modBaseAddr;
    public uint modBaseSize;
    public IntPtr hModule;
    public string szModule;
    public string szExePath;
}
```

- Поле *dwSize* определяет размер структуры и поэтому должно быть инициализировано значением `SizeOf (MODULEENTRY32)` до её использования.
- Поле *th32ModuleID* представляет собой идентификатор модуля, который имеет значение только для функций `TooHelp32`.
- Поле *th32ProcessID* содержит идентификатор (ID) опрашиваемого процесса. Этот ID можно использовать с другими функциями `Win32`.
- Поле *GblcntUsage* содержит глобальный счетчик ссылок данного модуля.
- Поле *ProccntUsage* содержит счетчик ссылок модуля в контексте процесса-владельца.
- Поле *modBaseAddr* представляет собой базовый адрес модуля в памяти. Это значение действительно только в контексте идентификатора процесса `th32ProcessID`.
- Поле *modBaseSize* определяет размер (в байтах) модуля в памяти.
- В поле *hModule* содержится дескриптор модуля. Это значение действительно только в контексте идентификатора процесса `th32ProcessID`.
- В поле *szModule* содержится строка с именем модуля, завершающаяся нуль-символом.
- Поле *szExepath* предназначено для хранения строки с ограничивающим нуль-символом, содержащей полный путь модуля.

Обработка информации о динамической памяти (кучах)

Опрос куч несколько сложнее опроса других типов объектов. В `TooHelp32` предусмотрены четыре функции, с помощью которых можно получить информацию о кучах. Первые две, `Heap32ListFirst()` и `Heap32ListNext()` позволяют выполнить проход по всем кучам процесса, а две другие `Heap32First()` и `Heap32Next()`, используются для получения более подробной информации обо всех блоках внутри отдельной кучи.

Функции `Heap32ListFirst()` и `Heap32ListNext()` определяются следующим образом:

```
public static extern bool Heap32ListFirst(IntPtr hSnapshot, ref HEAPLIST32 lph1);

public static extern bool Heap32ListNext(IntPtr hSnapshot, ref HEAPLIST32 lph1);
```

И вновь первый параметр является дескриптором снимка, а второй *lphe* представляет собой структуру типа **HEAPLIST32**, передаваемую по ссылке. Определение этой записи имеет следующий вид:

```
public struct HEAPLIST32
{
    public uint dwSize;
    public uint th32ProcessID;
    public uint th32HeapID;
    public uint dwFlags;
}
```

- Поле *dwSize* определяет размер структуры, и поэтому оно должно быть инициализировано значением `SizeOf()` до её использования.
- В поле *th32ProcessID* содержится идентификатор (ID) процесса-владельца.
- В поле *th32HeapID* содержится идентификатор (ID) кучи. Этот ID имеет значение только для заданного процесса, и его можно использовать только с функциями `ToolHelp32`.
- В поле *dwFlags* хранится признак, который определяет тип кучи. В качестве значения этого поля может использоваться либо константа `HF32_DEFAULT` (которая означает, что текущая куча является стандартной кучей процесса), либо константа `HF32_SHARED` (которая означает, что текущая куча является разделяемой обычным способом).

Функции `Heap32First()` и `Heap32Next()` определяются следующим образом:

```
public static extern bool Heap32First(IntPtr hSnapshot,
    ref HEAPENTRY32 lphe, uint th32ProcessID, uint th32HeapID);
```

```
public static extern bool Heap32Next(IntPtr hSnapshot,
    ref HEAPENTRY32 lphe);
```

Списки параметров этих функций немного отличаются от соответствующих списков функций, связанных с перечислением процессов, потоков, модулей. Эти функции предназначены для перечисления блоков данной кучи в данном процессе, а не некоторых свойств одного процесса. При вызове функции `Heap32First()` параметры, установленные в полях *th32ProcessID* и *th32HeapID*, должны быть равны значениям одноименных полей структуры **HEAPLIST32**, заполненной с помощью функций `Heap32ListFirst()` или `Heap32ListNext()`. Параметр *lphe* функций `Heap32First()` и `Heap32Next()` имеет тип **HEAPENTRY32**. Эта структура содержит информацию, относящуюся к блоку кучи, и ее определение имеет следующий вид:

```

public struct HEAPENTRY32
{
    public uint dwSize;
    public IntPtr hHandle;
    public uint dwAddress;
    public uint dwBlockSize;
    public uint dwFlags;
    public uint dwLockCount;
    public uint dwResvd;
    public uint th32ProcessID;
    public uint th32HeapID;
}

```

- Поле *dwSize* определяет размер структуры и поэтому должно быть инициализировано значением *SizeOf()* до её использования .
- В поле *hHandle* содержится дескриптор блока кучи.
- Поле *dwAddress* представляет собой линейный адрес начала блока кучи.
- В поле *dwBlockSize* содержится размер в байтах этого блока кучи.
- В поле *dwFlags* хранится признак, который определяет тип блока кучи. Это поле может иметь одно из значений:

LF32_FIXED – блок памяти имеет фиксированное местонахождение.

LF32_FREE – блок памяти не используется.

LF32_MOVEABLE – блок памяти можно перемещать.

- Поле *dwLockCount* представляет собой счетчик блокировок блока памяти. Это значение увеличивается на единицу при каждом вызове процессом функции *GlobalLock()* или *LocalLock()*.
- Поле *dwResvd* зарезервировано в данный момент и не должно использоваться.
- В поле *th32ProcessID* содержится идентификатор процесса-владельца.
- Поле *th32HeapID* является идентификатором кучи, которой принадлежит блок.

Поскольку до составления списка блоков кучи придется сначала составить список куч, код опроса блоков кучи немного сложнее. В приведенном ниже классе *WalkHeaps()* вложены *Heap32First()/Heap32Next()* внутри цикла *Heap32ListFirst()/Heap32ListNext()*.

```

using System;
using System.Drawing;
using System.Collections;
using System.ComponentModel;
using System.Windows.Forms;
using System.Data;
using System.Runtime.InteropServices;

```



```

class WalkHeaps
{
    [Flags]
    internal enum SnapshotFlags : uint
    {
        HeapList = 0x00000001,
        Process = 0x00000002,
        Thread = 0x00000004,
        Module = 0x00000008,
        Module32 = 0x00000010,
        Inherit = 0x80000000,
        All = 0x0000001F,
        NoHeaps = 0x40000000
    }

    [DllImport("kernel32", SetLastError = true, CharSet = CharSet.Auto)]
    private static extern bool Heap32ListFirst([In]IntPtr hSnapshot, ref
        HEAPLIST32 lph1);

    [DllImport("kernel32", SetLastError = true, CharSet = CharSet.Auto)]
    private static extern bool Heap32ListNext([In]IntPtr hSnapshot, ref
        HEAPLIST32 lph1);

    [DllImport("kernel32", SetLastError = true, CharSet = CharSet.Auto)]
    private static extern bool Heap32First(ref HEAPENTRY32 lphe, [In]uint
        th32ProcessID, [In]uint th32HeapID);

    [DllImport("kernel32", SetLastError = true, CharSet = CharSet.Auto)]
    private static extern bool Heap32Next(ref HEAPENTRY32 lphe);

    [DllImport("kernel32", SetLastError = true, CharSet = CharSet.Auto)]
    private static extern IntPtr CreateToolhelp32Snapshot([In]UInt32 dwFlags,
        [In]uint th32ProcessID);

    [DllImport("kernel32", SetLastError = true, CharSet = CharSet.Auto)]
    private static extern bool CloseToolhelp32Snapshot([In]IntPtr hSnapshot);

    [DllImport("kernel32", SetLastError = true, CharSet = CharSet.Auto)]
    private static extern bool CloseHandle([In]IntPtr hObject);

    public static IEnumerable<HEAPENTRY32> GetBlocks(uint pid)
    {
        List<HEAPENTRY32> blocks = new List<HEAPENTRY32>();
        IntPtr curSnap =
            CreateToolhelp32Snapshot((uint)SnapshotFlags.All, pid);
        try
        {
            HEAPLIST32 h1 = new HEAPLIST32();
            h1.dwSize = (UInt32)Marshal.SizeOf(typeof(HEAPLIST32));

            HEAPENTRY32 he = new HEAPENTRY32();
            he.dwSize = (UInt32)Marshal.SizeOf(typeof(HEAPENTRY32));

```

```

        if (Heap32ListFirst(curSnap, ref h1))
        {
            do
            {
                if (Heap32First(ref he, h1.th32ProcessID,
                    h1.th32HeapID))
                {
                    do
                    {
                        yield return he;
                    } while (Heap32Next(ref he));
                }
            } while (Heap32ListNext(curSnap, ref h1));
        }
    }
finally
{
    CloseHandle(curSnap);
}
}
}
}

```

Задание 3 для самостоятельного выполнения

1. Для каждого процесса, имеющего заданное число потоков, вывести идентификаторы этих потоков.
2. Из процессов с идентификаторами из заданного диапазона выбрать те, у которых модуль максимальной длины.
3. Выбрать процессы, у которых есть потоки из заданного диапазона идентификаторов.
4. Выбрать процесс, у которого родительский процесс максимальной длины.
5. Выбрать процессы, у которых есть потоки с заданным значением приоритета.
6. Вывести информацию о владельцах блоков, больших заданного n .
7. Вывести информацию о владельцах блоков из заданного диапазона линейных адресов.
8. Вывести информацию о владельцах самого большого объема свободной памяти.
9. Вывести информацию о владельцах самого большого объема фиксированной памяти.
10. Вывести информацию о владельцах самого большого числа блоков.
11. Вывести информацию о процессах, имеющих потоки с большим или меньшим, чем базовый приоритетом.
12. Вывести информацию о процессах, имеющих наибольшее число модулей.

13. Вывести информацию о процессах, имеющих наибольший суммарный размер модулей.
14. Вывести информацию о процессах, с наибольшим счетчиком ссылок.
15. Вывести информацию о процессах, у которых имя модуля соответствует заданной маске.
16. Вывести информацию о процессах, с минимальным размером модуля.
17. Для каждого процесса найти модуль наибольшей длины.
18. Определить, есть ли процесс с заданным количеством модулей.
19. Подсчитать количество процессов, у которых есть потоки с заданной величиной приоритета.
20. Для каждого процесса найти и вывести модуль максимального размера.

Список литературы

1. Таненбаум, Э. Современные операционные системы / Э. Таненбаум, Х. Бос. – 4-е изд. – СПб. : Питер, 2015. – 1120 с.
2. Основы современных операционных систем / В.О. Сафонов. — Москва : Интернет — Университет Информационных технологий, 2011, 584с. // “Университетская библиотека online” : URL : <http://biblioclub.ru>
3. Олифер, В.Г. Сетевые операционные системы : учебник для вузов / В. Г. Олифер; Н. А. Олифер .— 2-е изд. — Санкт-Петербург : Питер, 2002.— 668 с.
4. Карпов В.Е., Коньков К.А. Основы операционных систем Интернет-университет информационных технологий [электронный ресурс], – <http://www.intuit.ru/department/os/osintro/lit.html>